Bagging

2026-04-17

Introduction

Bagging — bootstrap aggregation, introduced by Breiman in 1996 — trains B models on B bootstrap samples of the training data and averages their predictions. The averaging dramatically reduces variance for high-variance, low-bias learners (deep decision trees, kNN with small k) without increasing their bias. Random forests are bagged trees with an additional feature-randomisation step that decorrelates the individual trees and amplifies the variance reduction.

Prerequisites

A working understanding of bootstrap resampling, the bias-variance trade-off, and decision trees as the canonical high-variance base learner.

Theory

For B predictions $\hat{y}_b$ each with variance σ^2 and pairwise correlation ρ, the variance of their mean is

$$\operatorname{Var}\left(\tfrac{1}{B}\sum_b \hat{y}_b\right) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B}.$$

As B grows, the second term vanishes; the first term is the residual variance limited by correlation. Bagging alone reduces variance via the $(1-\rho)/B$ contribution; adding decorrelation (feature subsampling, as in random forest) lowers ρ and produces further variance reduction.

Out-of-bag (OOB) observations — the roughly 37 % of rows not selected by each bootstrap — provide a free held-out validation set: for each row, average predictions from trees that did not include it during training.

Assumptions

The base learner benefits from variance reduction; bagging is most effective on high-variance learners (deep trees, kNN with small k, neural networks) and does essentially nothing for stable learners like linear regression.

R Implementation

```
library(ipred); library(rpart)

set.seed(2026)
n <- 400
df <- data.frame(
  x1 = rnorm(n), x2 = rnorm(n),
  y  = factor(ifelse(plogis(1 * rnorm(n) + 0.5 * rnorm(n)^2) > runif(n),
                     "yes", "no"))
)

# Bagged trees (no feature subsampling -- contrast with random forest)
```

```r
m <- bagging(y ~ ., data = df, nbagg = 100, coob = TRUE)
print(m)
```

Output & Results

`ipred::bagging()` returns a fitted bagged ensemble with the OOB misclassification rate when `coob = TRUE`. The OOB error is the standard generalisation estimate; comparing it to a single tree's CV error quantifies the variance reduction from bagging.

Interpretation

A reporting sentence: "100 bagged trees achieved OOB misclassification rate of 0.21, an improvement over a single tree's CV error of 0.28; switching to a random forest with feature subsampling reduced OOB further to 0.18, illustrating that decorrelation contributes additional variance reduction beyond plain bagging." Always compare bagged ensembles against single-base-learner baselines.

Practical Tips

- Bagging primarily benefits high-variance learners (trees, kNN with small k); linear models are stable and gain little.
- Use $B = 100$ to 500 bootstrap replicates; returns diminish quickly beyond that, and computation grows linearly.
- OOB error is nearly unbiased for moderate to large n and free to compute alongside training; prefer it to cross-validation for rapid iteration.
- Random forests (bagging + feature subsampling) almost always outperform plain bagged trees; reach for `ranger` directly rather than tuning `ipred::bagging`.
- For classification, average class probabilities (soft voting) rather than class labels (hard voting); soft voting is consistently more accurate.
- Bagging extends beyond trees: any learner can be bagged via `ipred::bagging` with `bf` set to a custom training function.

R Packages Used

`ipred::bagging()` for canonical bagging on arbitrary base learners; `ranger` and `randomForest` for bagged trees with feature subsampling (random forests); `parsnip::bag_tree()` for tidymodels integration; `mlr3pipelines::po("subsample")` for bagging within mlr3 pipelines.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro

- tidymodels pipelines
- FDR, knockoffs, replication crisis